ISIM
Institut Supérieur d'Informatique de Médenine

Année Universitaire
2025-2026

Travaux Pratiques - Développement Web Avancé

TP N°3 : Interfaces avec Blade

Classe : MP1ILC **Enseignant :** M. JARRAY Mondher **Durée :** 1h30 **Projet :** TaskFlow

Objectifs du TP

- Créer un layout principal pour TaskFlow
- Utiliser la syntaxe Blade pour afficher des données
- Créer des vues pour les projets et les tâches
- Implémenter des composants réutilisables
- Construire des formulaires avec protection CSRF

Prérequis

- Avoir terminé le TP 2 (routes et contrôleurs)
- Comprendre la syntaxe Blade (Chapitre 3)
- Projet TaskFlow fonctionnel

Partie 1 : Layout Principal (25 min)

Exercice 1.1 : Créer la structure des dossiers

Créez l'arborescence suivante dans `resources/views/` :

```
views/
    layouts/
        app.blade.php
    components/
        alert.blade.php
        card.blade.php
    partials/
        navbar.blade.php
        footer.blade.php
    projets/
        index.blade.php
        show.blade.php
        create.blade.php
    taches/
        index.blade.php
        create.blade.php
    welcome.blade.php
```

Créez ces dossiers et fichiers vides pour commencer.

Exercice 1.2 : Créer le layout principal

Créez le fichier `layouts/app.blade.php` avec le contenu suivant :

```blade
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>@yield('title', 'TaskFlow')</title>

    <!-- CSS simple pour le TP -->
    <style>
        * { margin: 0; padding: 0; box-sizing: border-box; }
        body { font-family: Arial, sans-serif; background: #f5f5f5; }
        .navbar { background: #2c3e50; padding: 1rem 2rem; }
        .navbar a { color: white; text-decoration: none; margin-right: 20px; }
        .navbar a:hover { text-decoration: underline; }
        .container { max-width: 1200px; margin: 2rem auto; padding: 0 1rem; }
        .card { background: white; border-radius: 8px; padding: 1.5rem;
                margin-bottom: 1rem; box-shadow: 0 2px 4px rgba(0,0,0,0.1); }
        .btn { padding: 0.5rem 1rem; border: none; border-radius: 4px;
               cursor: pointer; text-decoration: none; display: inline-block; }
        .btn-primary { background: #3498db; color: white; }
        .btn-success { background: #27ae60; color: white; }
        .btn-danger { background: #e74c3c; color: white; }
        .alert { padding: 1rem; border-radius: 4px; margin-bottom: 1rem; }
        .alert-success { background: #d4edda; color: #155724; }
        .alert-danger { background: #f8d7da; color: #721c24; }
        .form-group { margin-bottom: 1rem; }
        .form-group label { display: block; margin-bottom: 0.5rem; font-weight: bold; }
        .form-group input, .form-group textarea, .form-group select {
            width: 100%; padding: 0.5rem; border: 1px solid #ddd; border-radius: 4px;
        }
        footer { text-align: center; padding: 2rem; color: #666; margin-top: 2rem; }
        .badge { padding: 0.25rem 0.5rem; border-radius: 4px; font-size: 0.8rem; }
        .badge-success { background: #27ae60; color: white; }
        .badge-warning { background: #f39c12; color: white; }
        .badge-secondary { background: #95a5a6; color: white; }
    </style>

    @yield('styles')
</head>
<body>
    {{-- Navigation --}}
    @include('partials.navbar')

    {{-- Contenu principal --}}
    <main class="container">
        {{-- Messages flash --}}
        @if(session('success'))
            <div class="alert alert-success">
                {{ session('success') }}
            </div>
        @endif

        @if(session('error'))
            <div class="alert alert-danger">
                {{ session('error') }}
            </div>
        @endif

        @yield('content')
    </main>

    {{-- Pied de page --}}
    @include('partials.footer')
```

```
    @yield('scripts')
</body>
</html>
```

Exercice 1.3 : Créer les partiels

Fichier : partials/navbar.blade.php

```blade
<nav class="navbar">
    <a href="{{ route('home') }}" style="font-weight: bold; font-size: 1.2rem;">
        TaskFlow
    </a>
    <a href="{{ url('/projets') }}">Projets</a>
    <a href="{{ url('/taches') }}">Tâches</a>
</nav>
```

Fichier : partials/footer.blade.php

```blade
<footer>
    <p>© {{ date('Y') }} TaskFlow - ISIM Médenine</p>
    <p>Développé par la classe MP1ILC</p>
</footer>
```

Ajoutez aussi la route "home" dans web.php :

```php
Route::get('/', function () {
    return view('welcome');
})->name('home');
```

Partie 2 : Vues des Projets (30 min)

Exercice 2.1 : Liste des projets (index)

1. Modifiez ProjetController pour passer des données :

```php
public function index()
{
    // Données simulées (plus tard depuis la BDD)
    $projets = [
        (object) ['id' => 1, 'nom' => 'Site E-commerce',
                  'description' => 'Boutique en ligne', 'statut' => 'en_cours'],
        (object) ['id' => 2, 'nom' => 'Application Mobile',
                  'description' => 'App iOS/Android', 'statut' => 'termine'],
        (object) ['id' => 3, 'nom' => 'API REST',
                  'description' => 'Backend API', 'statut' => 'en_attente'],
    ];

    return view('projets.index', compact('projets'));
}
```

2. Créez la vue projets/index.blade.php :

```blade
@extends('layouts.app')

@section('title', 'Mes Projets')

@section('content')
    <div style="display: flex; justify-content: space-between; align-items: center; margin-bottom: 1.5rem;
        <h1>Mes Projets</h1>
        <a href="{{ url('/projets/create') }}" class="btn btn-primary">
            + Nouveau Projet
        </a>
    </div>

    @forelse($projets as $projet)
        <div class="card">
            <div style="display: flex; justify-content: space-between;">
                <div>
                    <h3>{{ $projet->nom }}</h3>
                    <p>{{ $projet->description }}</p>
                </div>
                <div>
                    @if($projet->statut === 'termine')
                        <span class="badge badge-success">Terminé</span>
                    @elseif($projet->statut === 'en_cours')
                        <span class="badge badge-warning">En cours</span>
                    @else
                        <span class="badge badge-secondary">En attente</span>
                    @endif
                </div>
            </div>
            <div style="margin-top: 1rem;">
                <a href="{{ url('/projets/' . $projet->id) }}" class="btn btn-primary">
                    Voir
                </a>
            </div>
        </div>
    @empty
        <div class="card">
            <p>Aucun projet trouvé. Créez votre premier projet !</p>
        </div>
    @endforelse
@endsection
```

Capture d'écran : Page liste des projets

Exercice 2.2 : Détail d'un projet (show)

1. Modifiez la méthode show() du contrôleur :

```php
public function show($id)
{
    // Simulation d'un projet avec ses tâches
    $projet = (object) [
        'id' => $id,
        'nom' => 'Site E-commerce',
        'description' => 'Développement d\'une boutique en ligne complète',
        'statut' => 'en_cours',
        'date_creation' => '2025-01-15',
    ];

    $taches = [
        (object) ['id' => 1, 'titre' => 'Créer la BDD', 'statut' => 'termine'],
        (object) ['id' => 2, 'titre' => 'Design UI', 'statut' => 'en_cours'],
        (object) ['id' => 3, 'titre' => 'Intégration paiement', 'statut' => 'en_attente'],
    ];

    return view('projets.show', compact('projet', 'taches'));
}
```

2. Créez la vue projets/show.blade.php :

La vue doit afficher :

- **a** Le nom et la description du projet
- **b** Le statut avec un badge coloré
- **c** La liste des tâches du projet
- **d** Un bouton "Retour" vers la liste

Écrivez votre code pour projets/show.blade.php :

Exercice 2.3 : Formulaire de création (create)

Créez `projets/create.blade.php` avec un formulaire :

```blade
@extends('layouts.app')

@section('title', 'Nouveau Projet')

@section('content')
    <h1>Créer un nouveau projet</h1>

    <div class="card">
        <form action="{{ url('/projets') }}" method="POST">
            @csrf

            <div class="form-group">
                <label for="nom">Nom du projet *</label>
                <input type="text" id="nom" name="nom" required
                       placeholder="Ex: Site E-commerce">
            </div>

            <div class="form-group">
                <label for="description">Description</label>
                <textarea id="description" name="description" rows="4"
                          placeholder="Décrivez votre projet..."></textarea>
            </div>

            <div class="form-group">
                <label for="statut">Statut</label>
                <select id="statut" name="statut">
                    <option value="en_attente">En attente</option>
                    <option value="en_cours">En cours</option>
                    <option value="termine">Terminé</option>
                </select>
            </div>

            <div style="margin-top: 1.5rem;">
                <button type="submit" class="btn btn-success">Créer le projet</button>
                <a href="{{ url('/projets') }}" class="btn" style="background: #ddd;">
                    Annuler
                </a>
            </div>
        </form>
    </div>
@endsection
```

Modifiez la méthode store() pour tester :

```php
public function store(Request $request)
{
    // Afficher les données reçues (temporaire)
    return response()->json([
        'message' => 'Projet reçu !',
        'data' => $request->all()
    ]);
}
```

Question : Que se passe-t-il si vous retirez `@csrf` du formulaire ?

Partie 3 : Composants et Améliorations (20 min)

Exercice 3.1 : Créer un composant Card

Créez `components/card.blade.php` :

```blade
@props(['title' => null])

<div class="card">
    @if($title)
        <h3 style="margin-bottom: 1rem;">{{ $title }}</h3>
    @endif
    {{ $slot }}
</div>
```

Utilisez-le dans vos vues :

```blade
<x-card title="Mon Projet">
    <p>Contenu de la carte...</p>
</x-card>

<x-card>
    <p>Carte sans titre</p>
</x-card>
```

Exercice : Refactorisez la page projets/index.blade.php pour utiliser ce composant.

Exercice 3.2 : Composant Badge de Statut

Créez un composant `components/status-badge.blade.php` qui affiche automatiquement le bon badge selon le statut :

Écrivez le code du composant :

Utilisation souhaitée :

```blade
<x-status-badge status="en_cours" />
<x-status-badge status="termine" />
<x-status-badge status="en_attente" />
```

Exercice 3.3 : Page d'accueil améliorée

Modifiez `welcome.blade.php` pour créer une vraie page d'accueil avec :

- **a** Un titre "Bienvenue sur TaskFlow"
- **b** Une description de l'application
- **c** Deux boutons : "Voir les projets" et "Créer un projet"
- **d** Utilisation du layout principal

Capture d'écran : Page d'accueil

Barème d'évaluation

Structure des dossiers (1.1)	Layout principal (1.2)
Partiels navbar/footer (1.3)	Liste des projets (2.1)
Détail projet (2.2)	Formulaire création (2.3)
Composant Card (3.1)	Composant Badge (3.2)
Page d'accueil (3.3)	

Total

☆ Exercice Bonus

1. Ajoutez un système de pagination simulé (afficher "Page 1 sur 3")
2. Créez une vue pour éditer un projet (projets/edit.blade.php)
3. Ajoutez des icônes (Unicode ou Font Awesome) dans l'interface

Livrables à rendre

- Tous les fichiers du dossier views/
- Le fichier ProjetController.php modifié
- Captures d'écran de toutes les pages
- Réponses aux questions